# 25.09 — Представления VIEW

## Конспект

VIEW - именованный запрос, определение которого хранится в базе. Обычное представление не содержит отдельную копию строк; данные получаются из исходных таблиц при обращении.

Представление создают через CREATE VIEW ... AS SELECT ..., изменяют через ALTER VIEW, удаляют через DROP VIEW. Создание представления начинают в отдельном пакете.

В запросе представления допустимы JOIN, агрегатные и оконные функции. Обычное VIEW не принимает параметры. Сортировку результата задают во внешнем запросе через ORDER BY.

VIEW удобно для повторного использования запросов, отчётов и ограничения видимых столбцов или строк. Для ограничения доступа нужно настроить права. Само создание VIEW не гарантирует ускорение.

## Ответы на вопросы для самопроверки

1. Что такое VIEW?
VIEW - объект базы, определённый SQL-запросом.

2. Чем представление отличается от таблицы?
Таблица хранит строки, обычное представление хранит определение запроса.

3. Где хранится определение обычного VIEW?
В метаданных базы; определение можно посмотреть через sys.sql_modules или OBJECT_DEFINITION.

4. Как создать представление?
CREATE VIEW имя AS SELECT ...;

5. Как изменить существующее представление?
ALTER VIEW имя AS SELECT ...;

6. Как удалить представление?
DROP VIEW имя;

7. Можно ли использовать JOIN внутри VIEW?
Да, JOIN допустим.

8. Можно ли использовать агрегатные функции внутри представления?
Да, например COUNT, AVG, MIN и MAX.

9. Можно ли использовать оконные функции внутри VIEW?
Да, оконные функции допустимы.

10. Почему VIEW не следует считать копией таблицы?
При обращении данные берутся из исходных таблиц, а отдельная копия строк не хранится.

11. Почему создание VIEW само по себе не гарантирует ускорения запроса?
Обычный VIEW не материализует результат: сервер всё равно выполняет запрос к исходным данным.

12. В каких ситуациях представление удобно использовать?
Для отчётов, повторного использования запросов и удобного доступа к части данных.

13. Чем VIEW отличается от хранимой процедуры?
VIEW используется как источник запроса, процедура запускается через EXEC и может выполнять несколько действий.

14. Чем VIEW отличается от пользовательской функции?
Функция может принимать параметры; обычное VIEW параметров не принимает.

15. Как представление может участвовать в ограничении доступа к данным?
Можно дать права на представление, скрывающее часть столбцов или строк, и не давать прямые права на исходные таблицы.

## Практическая часть

## Подготовка учебных оценок

Продолжение учебной базы предыдущих заданий. Если таблица grades уже создана и заполнена, подготовку пропустить. Средний балл в представлении вычисляется по grades, а не берётся из students.average_grade.

```
USE CollegeDB;
GO
CREATE TABLE college.grades
(
    id INT IDENTITY(1,1) PRIMARY KEY,
    student_id INT NOT NULL,
    grade INT NOT NULL CHECK (grade BETWEEN 2 AND 5),
    FOREIGN KEY (student_id) REFERENCES college.students(id)
);
INSERT INTO college.grades (student_id, grade)
VALUES (1, 5), (1, 4), (2, 4), (2, 5), (3, 3),
       (3, 4), (4, 5), (4, 5), (5, 2), (5, 3);
GO
```

## Создание student_statistics

LEFT JOIN сохраняет студентов без оценок: количество равно 0, среднее, минимум и максимум равны NULL. Приведение grade к DECIMAL позволяет получить дробное среднее.

```
CREATE VIEW college.student_statistics
AS
SELECT s.id AS student_id, s.name AS student_name,
    g.name AS group_name,
    COUNT(gr.id) AS grade_count,
    AVG(CAST(gr.grade AS DECIMAL(4,2))) AS average_grade,
    MIN(gr.grade) AS min_grade,
    MAX(gr.grade) AS max_grade
FROM college.students s
LEFT JOIN college.groups g ON g.id = s.group_id
LEFT JOIN college.grades gr ON gr.student_id = s.id
GROUP BY s.id, s.name, g.name;
GO
```

## Запросы и добавление оценки

До добавления у Ивана 2 оценки, среднее 4.5, минимум 4, максимум 5. После добавления оценки 5: количество 3, среднее примерно 4.6667, минимум и максимум не изменились. VIEW пересоздавать не нужно.

```
SELECT * FROM college.student_statistics;

SELECT * FROM college.student_statistics
WHERE average_grade >= 4.5;

SELECT * FROM college.student_statistics
ORDER BY average_grade DESC;

SELECT * FROM college.student_statistics
WHERE group_name = N'ИС-31';

SELECT * FROM college.student_statistics WHERE student_id = 1;
INSERT INTO college.grades (student_id, grade) VALUES (1, 5);
SELECT * FROM college.student_statistics WHERE student_id = 1;
```